Week 05: two kinds of work

Programming task

Session 2: three short Python tasks.
Predict, change and repair.
Session 3: separate application.

Research task

Share Research 3 in Session 1.
After class: Research 4.
Discuss in Week 6 · proposed 5%.

Use this guide with the same Week notebook; do not create a second set of weekly tasks. Email: submit.progmis@gmail.com.

Programming practice

Text conversion, validation and try/except
Predict → run → change → repair → explain

Three core tasks: understand the Python

Inspect text and int

Compare "4" and int("4"): predict values, types and products.

Change and handle

Try "4", "0", "-1", "many". Trace conversion, check and except.

Repair validation

A < 0 check accepts zero. Fix it; preserve valid and bad-text cases.

Open Week_05.ipynb → Session 2. Keep a prediction, result and explanation for each task.

Core 1–2: predict, then change one thing

```
raw = "4"
print(raw, type(raw).__name__)
count = int(raw)
print(count, type(count).__name__)
print(raw * 2)
print(count * 2)
```

1. Predict types: str, then int. Products: 44, then 8.
2. In Core 2, try 4, 0, -1 and many as text.
3. Separate conversion failure from a rejected range.

Use the complete Core 1 and Core 2 starters in the notebook; explain each changed result.

Core 3: find and repair one mistake

```
def parse_count(raw):
    count = int(raw)
    if count < 0:  # Deliberate range-check bug.
        raise ValueError("count must be positive")
    return count
for raw in ["4", "0", "many"]:
    try:
        print(raw, parse_count(raw))
    except ValueError:
        print(raw, "rejected")
```

1. Positive means greater than zero.
2. Repair the zero case; keep valid text working.
3. Explain why many fails before the range check.

Edit the matching Core 3 cell. Keep the wrong result, your repair and a retest.

Done: explain the concept, then save

Step	What to do / keep
Predict	Record string/int types and the different products.
Change	Explain conversion versus range failure for four inputs.
Repair	Reject zero; keep 4 valid and catch many.
Optional	Compare 2 / 0 / many in the supplied CLI.
Save	Keep code and three records in Week_05.ipynb.

After practice, explain conversion, range checks and the except path.

Project application

Continue in the same notebook after the Python practice.

Apply the idea: validate the six-row CSV

Step	What to do / keep
Open	Project studio → One small implementation task: validate a row.
Edit	validate_offer_row: ID, int conversion, positive counts, copy.
Core checks	Valid row returns integers; many raises ValueError.
Run helper	build_import_audit(offer_file, validate_offer_row).
Explain / save	Two proposed, two held, two rejected; reasons and next actions.

Use the supplied grouping helper. CLI, reordered rows and a custom grouping algorithm stay optional.

Research how programs keep data

What to research

Find out how files or databases let a program retrieve information after it closes.

Possible directions

Look at record IDs, saved fields and retrieval. Consider a fact whose original value should be kept even after a later correction.

A short summary example

Plan EX-8 saved 2 containers at 4 portions each. If the current unit becomes 5, keeping the original value still explains the old total of 8.

Discuss in Week 6 · summarize in your own words and cite your sources.